```
def create(conn, params) do
  data = DataStore.persist!(params)
  EnrichWorker.enqueue!(data, params)
  json(conn, %{id: data.id})
end

# Example of an Oban Worker

defmodule EnrichWorker do
  use Oban.Worker

  def enqueue!(data, params) do
    %{id: data.id, params: params}
    |> new()
    |> Oban.insert!()
  end

  def perform(%Oban.Job{args: %{"id" => data_id, "params" => params}}) do
    enriched = enrich_data_via_api!(params)
    DataStore.update_enriched!(data_id, enriched)
    :ok
  end
end
```

You can see from this example that Oban workers are modules that implement the perform/1 function and they use Oban.Worker. That's all you need to create an Oban worker. You don't need to know anything about the underlying process model to use it, but you benefit from OTP best practices without even thinking about it!

We need to consider the potential downsides of this approach. There is slightly more code than the alternatives, but it's actually quite reasonable. The biggest downside is that a Postgres row is inserted and processed for each job placed into the queue. You won't run into a problem with this unless you're processing a lot of jobs per second, but it's worth mentioning. (Later in this chapter, we'll talk about why Postgres is actually a great choice for a job system.)

Overall, Oban is an excellent choice for asynchronous jobs. If you're building a system and looking to run code asynchronously, save yourself the hassle and start with Oban.